

HelixSkills

HelixSkills

Канстытуцыйна забяспечаная сістэма навыкаў для агентаў CLI AI.

Зводка

HelixSkills — гэта сістэма навыкаў для агентаў CLI AI, якая ўключае Helix Constitution як падмадуль, таму ўсе ўніверсальныя правілы кіравання прымяняюцца безумоўна. Яна аб'ядноўвае ўсталёўваемыя навыкі агентаў, серверы інструментаў MCP, плагіны кода Claude і паўторна выкарыстоўваемыя рухавікі ў рэгістраваны і дакументаваны каталог.

Кароткае апісанне

HelixSkills — гэта сістэма навыкаў для агентаў CLI AI. Яна ўбудоввае Helix Constitution як падмадуль, каб усе ўніверсальныя правілы дзейнічалі аўтаматычна, а затым пастаўляе рэгістравальныя навыкі (прэфікс дзеянняў, валідатар медыя, мультытрэк, сінхранізацыя сеансаў, жыццёвы цыкл працаздольнага элемента і іншыя), два серверы інструментаў MCP, два плагіны кода Claude і паўторна выкарыстоўваемыя рухавікі.

Падрабязнае апісанне

HelixSkills (рэпазіторый `skills`, ліцэнзія Apache-2.0) — гэта сістэма навыкаў для агентаў CLI AI, якая пачынаецца з наўмыснай інверсіі звычайнага парадку: кіраванне стаіць на першым месцы, магчымасці — на другім. Яна ўключае Helix Constitution як падмадуль `constitution/`, таму ўсе ўніверсальныя правілы з файлаў `constitution/CLAUDE.md` і `constitution/Constitution.md` прымяняюцца безумоўна — не як дамоўленасць, якую агент можа паважаць, а як набор

правілаў, фізічна ўбудаваных у структуру праекта. Агент, які прымае HelixSkills, не можа адмовіцца ад канстытуцыі: правілы перамяшчаюцца разам з кодам.

У той час як большасць «фрэймворкаў навыкаў» працуюць з абстракцыямі, HelixSkills пастаўляе канкрэтны, рэгістравальны інвентар, на які можна паказаць і ўсталяваць. Сямі канстытуцыйных навыкаў усталёўваюцца праз `register.sh`: сістэма прэфіксаў дзеянняў, валідатар медыя, мультытрэк, справаздача па працаздольных элементах, чарга запланаваных работ, сінхранізацыя сеансаў і жыццёвы цыкл працаздольнага элемента — спектр ад прамежкавага да прасунутага ўзроўню, які ахоплівае ўсё: ад дысцыплінаванага наймення дзеянняў да валідацыі медыя і поўнага жыццёвага цыклу адзінкі працы. Дадатковыя чарнавыя навыкі (агляд Android, мова Java/Kotlin, аперацыйная сістэма Linux) ужо індэксаваны і падрыхтаваны да актывацыі. Два серверы інструментаў MCP (валідатар медыя, запланаваныя работы) падаюць гэтыя навыкі агентам праз Model Context Protocol, а два плагіны кода Claude (`helix`, `scheduled-work`) убудовуюць тыя ж магчымасці непасрэдна ў асяроддзе агента — адзін набор навыкаў, які дасягае агентаў праз розныя інтэрфейсы.

Пад каталогам знаходзяцца чатыры паўторна выкарыстоўваемыя рухавікі першага ўзроўню — `continuum` (рэалізаваны), а таксама `session_orchestrator`, `token_optimizer` і `clickup_sync` (на стадыі распрацоўкі) — агульная інфраструктура, якая вызваляе навыкі ад неабходнасці паўторна ствараць адны і тыя ж механізмы. Адзіны `token_optimizer` дэкларуе відавочны граф залежнасцей, які звязвае яго з пакетамі экасістэмы `vasic-digital` (TOON, Embeddings, VectorDB, Normalize, conversation) і HelixDevelopment LLMProvider, таму яго міжрэпазітарныя сувязі можна праверыць, а не здагадацца пра іх. Усё гэта ахоплена дысцыплінаванай дакументацыяй: каталог навыкаў, аўтаматычна згенераваны індэкс графа навыкаў, дэталёвыя старонкі па рэпазіторыях і адкрыты рэестр прабел і рызык, дзе пералічана, што яшчэ не зроблена. Уся сістэма дублюецца на GitHub, GitLab, GitFlic і GitVerse для забеспячэння ўстойлівасці і рэгіянальнага доступу.

Змест

Чаму мы яго стварылі

Агентам CLI і AI патрэбны магчымасці, якія з’яўляюцца паслядоўнымі, рэгламентаванымі і шматразовымі — а не адмысловыя скрыпты, дзе кожны раз перавынаходзяцца правілы. HelixSkills быў створаны, каб надаць агентам упакаваны, рэгістравальны набор навыкаў, які звязаны з

агульнай канстытуцыяй, каб паводзіны заставаліся паслядоўнымі і падсправаздачнымі ў кожнага агента і праекта, што яго прымае.

Чаму гэта змяняе правілы гульні

Гэта робіць магчымасці агента пераноснымі і адпаведнымі правілам *па канструкцыі*, а не па дысцыпліне. Кожны навык — гэта кіраваны, версіяваны, усталявальны модуль, які падтрымліваецца канстытуцыйным падмадулем, таму ў момант рэгістрацыі навыка агент таксама атрымлівае кананічны набор правіл без магчымасці адхіленняў. Гэта адкрывае магчымасць, якая раней была непрактычнай: перанос магчымасці ад аднаго агента ці праекта да іншага з упэўненасцю, што яна прыйдзе ўжо звязанай з аднолькавым кіраваннем, даступнай праз стандартныя інтэрфейсы (серверы MCP і плагіны Claude Code), а не праз кучу спецыяльна напісаных скрыптаў, якія кожны раз перавынаходзяць правілы.

Што новага

- **Constitution як падмадуль:** універсальныя правілы кіравання наследуюцца, а не капіюцца — мацуюцца ў дрэва, каб кожны агент, які іх выкарыстоўвае, падпарадкоўваўся аднаму кананічнаму набору правіл, а абнаўленні паступалі з адзінай крыніцы ісціны замест дзясяткаў састарэлых копіяў.
- Навыкі пастаўляюцца як самарэгістравальныя адзінкі (`register.sh`) і ўбудовваюцца ў аўтаматычна згенераваны індэкс графа навыкаў, каб каталог заставаўся зручным для пошуку і ніколі не разыходзіўся з тым, што фактычна ўсталявана.
- Мультыінтэрфейсная даступнасць: аднолькавы набор навыкаў дасягае агентаў праз серверы інструментаў MCP і плагіны Claude Code — напішы адзін раз, звяртайся да любой асяроддзя выканання, якое выкарыстоўвае агент.
- Шматразовыя рухавікі першага ўзроўню (`continuum`, `token_optimizer`, `session_orchestrator`, `clickup_sync`), агульныя для ўсяго экасістэмы, кожны з якіх нясе ў сабе відавочныя, падсправаздачныя дэкларацыі залежнасцей паміж рэпазіторыямі замест схаванага звязвання.

Галоўныя тэхнічныя выклікі і як мы іх пераадолені

- **Захаванне паслядоўнасці паводзін агентаў і адпаведнасці правілам у шматлікіх навыках і агентах**

— паўторная рэалізацыя кіравання для кожнага навыка гарантуе разыходжанне з часам. Вырашана шляхам мацавання Helix Constitution як падмадуля, каб правілы ў constitution/CLAUDE.md і constitution/Constitution.md прымяняліся безумоўна і абнаўляліся з адзінай крыніцы, а не капіяваліся і пакідаліся на самацек.

- **Зрабіць пашыральны набор навыкаў усталявальным і зручным для пошуку** — каталог марны, калі ніхто не можа знайсці ці ўсталяваць тое, што ў ім ёсць. Вырашана з дапамогай рэгістрацыі кожнага навыка праз register.sh, якая падключае навык пры ўсталяванні, а таксама аўтаматычна згенераванага індэкса графа навыкаў INDEX і дэталёвай дакументацыі па кожным рэпазіторыі, каб пошук аўтаматычна адпавядаў рэальнаму стану.
- **Дасягненне агентаў, якія выкарыстоўваюць розныя асяроддзі выканання** — адну і тую ж магчымасць не трэба перабудоўваць для кожнага хоста. Вырашана шляхам упакоўкі аднаго набору навыкаў як праз азначэнні сервераў інструментаў MCP (у constitution/mcp/), так і праз плагіны Claude Code (у constitution/plugins/), каб адна рэалізацыя была даступнай праз усе інтэрфейсы.

Змест

Тэхналагічны стэк

- **Shell (асноўная мова)** — абраная таму, што інструменты ўстаноўкі і рэгістрацыі павінны працаваць усюды, дзе існуе агент, без папярэдняй загрузкі рантайму; на ёй напісаныя register.sh і install_upstreams, што забяспечвае свабоду ад залежнасцей і пераноснасць на старце.
- **Git-падмадулі** — абраныя для атрымання кіравання без дублявання: Helix Constitution падключаны ў каталог constitution/ як жывы дакумент, таму абнаўленні правіл распаўсюджваюцца праз адзін указальнік замест капіравання і забыцця.
- **Model Context Protocol (MCP)** — абраны як стандартны, незалежны ад рантайму інтэрфейс для агентаў; два MCP-серверы (media-validator, scheduled-work) вызначаны ў constitution/mcp/, каб падаваць навыкі як выклікаемыя інструменты.
- **Плагіны Claude Code** — абраныя для натуральнага ўбудавання навыкаў у рантайм агента без дадатковых злучэнняў; два плагіны (helix, scheduled-work) пастаўляюцца ў constitution/plugins/, дублюючы паверхню MCP для іншага хоста.
- **Перавыкарыстоўныя рухавічкі (continuum, token_optimizer, session_orchestrator, clickup_sync)** — абраныя для вынясення агульнай логікі з асобных навыкаў дзеля перавыкарыстання паміж праектамі; напрыклад,

token_optimizer падлучаны да пакетаў vasic-digital (TOON, Embeddings, VectorDB, Normalize, conversation) і HelixDevelopment LLMProvider праз дэклараваныя залежнасці, а не праз дубляванне кода.

- **Мнагахоставаем люстэркаванне Git (GitHub, GitLab, GitFlic, GitVerse)** — абранае для таго, каб адмова аднаго хоста ці рэгіянальная блакада не перапынілі доступ; адзін і той жа рэпазіторый падтрымліваецца ў працоўным стане на чатырох платформах дзеля надзейнасці і даступнасці.

Стан і заўвагі аб шчырасці

- **Стан: бэта.** Сямі канстытуцыйных навыкаў, два MCP-серверы і два плагіны ўжо пастаўлены; чарнавыя навыкі праіндэксаваны і чакаюць актывацыі, а тры з чатырох рухавічкоў першага ўзроўню (session_orchestrator, token_optimizer, clickup_sync) яшчэ знаходзяцца на стадыі распрацоўкі.
- У README праект завецца helix_skills; кананічны шлях на GitHub — [HelixDevelopment/skills](https://github.com/HelixDevelopment/skills). Колькасць выяўленых праблем у README падаецца як самаацэнка.

Прыярытэтны ўзровень: Helix-першасны.